

Contents

Specification Review Report	1
1. Executive Summary	1
2. Overall Maturity	2
3. Findings Summary	2
4. Detailed Findings	3
F-001 — Per-drill expected verdicts not pinned	3
F-002 — Synthetic surrogate formulas not pinned	3
F-003 — Decision wire format ambiguous	4
F-004 — MockPolicy script semantics under-specified	4
F-005 — <code>run_id</code> hash inputs unspecified	5
F-006 — Contradiction-detection protocol unspecified	5
F-007 — <code>repair_success</code> pairing rule ambiguous	6
F-008 — <code>drill --real</code> surface unmotivated	6
F-009 — Exit-code precedence unstated	6
F-010 — <code>seen_actions</code> bookkeeping gaps	7
F-011 — O-1/O-2 optionality ids unexplained	7
F-012 — Drill-report text fields' provenance unstated	7
5. Requirements Review	7
6. Interface and Data-Contract Review	8
7. State and Failure Review	8
8. Determinism and Algorithm Review	8
9. Edge-Case Review	8
10. Non-Functional Requirement Review	8
11. Security and Trust-Boundary Review	8
12. Observability and Provenance Review	8
13. Testing and Verification Review	8
14. Metrics and Evaluation Review	9
15. Traceability Review	9
16. Internal-Consistency Review	9
17. Architecture Review	9
18. Implementation-Agent Readiness	9
19. Quality Scorecard	9
20. Remediation Plan	10
P0 — Blocking (must resolve before implementation)	10
P1 — Important (resolve before claiming conformance)	10
P2 — Improvement (may defer)	10
21. Final Verdict	10

Specification Review Report

Subject: labs/week1/chapter5/SPEC.md (v0.1) — Bounded Research-Agent Runtime **Reviewer**
method: spec-review skill, four passes (comprehension → local precision → cross-consistency → implementation simulation)

1. Executive Summary

The specification is a disciplined, highly traceable Level 2 document that successfully carries the ch1–ch4 conventions (schema gates, E-13 availability taxonomy, offline mock doubles, source scan) onto a genuinely new surface — the agentic loop. Its strongest assets are the closed stopping-condition set, the total retry

taxonomy, the authorization-outside-the-model boundary, and complete acceptance coverage of all ten §34 drills.

Its weaknesses are concentrated on the **agent-specific surfaces that did not exist in prior chapters**: the decision wire format is a pseudo-schema rather than a pin, the token/latency/cost surrogate formulas are unnamed, the `MockPolicy` script semantics are sketched, and the per-drill expected verdicts are hedged in one place while declared pinned in another. None of these are structural; all are pinning gaps that can be closed inline.

Findings: 13 total — 0 CRITICAL / 3 HIGH / 6 MEDIUM / 4 LOW.

Maturity: **Level 2** (implementable with material ambiguities). Readiness: **READY WITH MINOR FIXES** — the HIGH findings are semantic pins, not redesigns.

2. Overall Maturity

Level 2 — Implementable. A competent engineer can implement the system end-to-end on the mock path, but three semantic contracts must be pinned before two independent implementers can produce byte-identical artifacts and before the drill harness can grade honestly across implementations. Level 3 is within reach once P0 findings are integrated.

3. Findings Summary

ID	Severity	Summary
F-001	HIGH	Per-drill expected termination/verdict table missing; T-08a hedges (“or documented fallback”) contra C-12 “pinned per drill”
F-002	HIGH	Token/latency/cost surrogate formulas for the synthetic mock path are not pinned
F-003	HIGH	Decision wire format ambiguous: type -discriminator vs sibling keys; final payload shape
F-004	MEDIUM	<code>MockPolicy</code> script semantics unpinned (top-hit selection, repair behavior, reasoning-entry text)
F-005	MEDIUM	<code>run_id</code> “deterministic content hash” inputs unspecified
F-006	MEDIUM	<code>contradiction_pair</code> contradiction-detection protocol unspecified (E-08/T-08f detection rule)
F-007	MEDIUM	<code>repair_success</code> numerator pairing rule ambiguous
F-008	LOW	<code>drill</code> advertises <code>--real</code> /exit 4 although grading is <code>MockPolicy</code> -only (R-09)
F-009	LOW	Exit-code precedence when error classes co-occur (usage vs corpus vs drill verdict)
F-010	LOW	<code>seen_actions</code> bookkeeping unclear for final , denied, and invalid decisions
F-011	LOW	O-1/O-2 optionality ids used without a taxonomy note

ID	Severity	Summary
F-012	MEDIUM	C-12 four-question text fields’ provenance (pinned template vs generated) unstated — byte-identity risk

4. Detailed Findings

F-001 — Per-drill expected verdicts not pinned

Severity: HIGH

Location: C-12, T-08a

Observation

C-12 requires `expected_*` fields to be “pinned per drill in `drills.py`”, but no table in the spec enumerates the expected `termination_reason` (and pass condition) for each of the ten drills, and T-08a itself reads “expected termination `goal_complete` (or documented fallback)”.

Why it matters

A drill’s verdict is the acceptance signal for the entire §34 exercise. Two implementers choosing different expectations produce different `pass/fail` outcomes for the same run.

Potential consequence

T-08a...T-08j and T-09 (honest-grading meta-test) cannot be shared across implementations; the drill harness grades whimsically.

Recommended resolution

Extend the C-11 FaultSpec table (or add a C-12a table) with a per-drill **expected termination_reason** and the exact pass comparison, e.g. `search_timeout` → `goal_complete`, `empty_results` → `goal_complete` (`insufficient_evidence`), `infinite_loop` → `repeated_state`, `max_steps_exhaustion` → `max_steps`, and remove the “(or documented fallback)” hedge in T-08a.

F-002 — Synthetic surrogate formulas not pinned

Severity: HIGH

Location: R-13, I-003, C-04 (`tokens_used`, `cost_usd`, `started_monotonic`)

Observation

I-003 mandates byte-identity and labels figures `synthetic`, but the derivation formula is only described as “content-derived surrogates ... derived from content lengths” (R-13). No formula (e.g. `est_tokens(text) = ceil(len(text)/4)`) is adopted.

Why it matters

Byte-identity (tested by T-03b via `diff`) is meaningless as a conformance target across implementations if the surrogate numbers are implementer-chosen. `ch3` pinned its `MockEmbedder` (documented hashed-BoW vector) for exactly this reason; the surrogate here needs the same treatment.

Potential consequence

Two correct implementations emit different `trace.json` numerics; cross-implementation `diff` fails and the `usage_kind: synthetic` label stops meaning “reproducible”.

Recommended resolution

Adopt explicit formulas in C-04 or C-10, e.g. `tokens_used := sum over entries of ceil(len(serialized entry)/4)`, `latency_ms := deterministic hash-of-content figure`, `cost_usd := tokens_used × pinned price table`, and keep the free-text values out of the graded fields.

F-003 — Decision wire format ambiguous

Severity: HIGH

Location: §1 (Policy actor), C-03 (DECISION_SCHEMA), E-07

Observation

The Policy actor row describes decisions as `{type: "tool_call", tool, arguments} / {type: "final", report}`, but C-03’s pseudo-schema uses the siblings-as-keys form `{"tool_call": {...}} / {"final": {...}}`, and the curriculum §5 loop uses `decision.type / decision.answer`. E-07’s “wrong-type decision” handler presumes a canonical shape that is nowhere pinned.

Why it matters

The decision object is the *entire output language* of the policy (real and mock). Its lexical form is the one contract that must be exact.

Potential consequence

MockPolicy and OllamaPolicy disagree on serialization; validators written against one form reject the other; the E-07 error path cannot be shared.

Recommended resolution

Pin one canonical JSON shape in C-03 with a literal example, e.g. `{"type": "tool_call", "tool": "search", "arguments": {...}}` and `{"type": "final", "report": {...}}`, and mark the `type` field as the sole discriminator.

F-004 — MockPolicy script semantics under-specified

Severity: MEDIUM

Location: §1 (Policy actor), T-04, C-07 (`reasoning` entries)

Observation

The MockPolicy is “a documented, input-determined rule script (`search` → `retrieve top hit` → `finalize`)” but: (a) *top hit* is defined only via `search`’s ranking (“lexical overlap, ties broken by `doc_id` sort” — formula itself loose); (b) its **repair behavior** after an `invalid_arguments` observation (required by T-04) is unscripted; (c) **reasoning** step-entry text is free-form — I-003 byte-identity then demands fixed templates, which are not named.

Why it matters

The mock path is the CI surface; anything it emits becomes part of the byte-identity contract.

Potential consequence

Implementations diverge in repair steps and reasoning text; T-03b/T-04 outcomes drift.

Recommended resolution

Add a short §4 annex (or expand C-05) enumerating the MockPolicy rule list verbatim: decision sequence, the canonical repair re-issue (re-emit same tool with the fault’s canonical argument), and the fixed reasoning strings per step.

F-005 — `run_id` hash inputs unspecified

Severity: MEDIUM

Location: C-07, I-003, K-05

Observation

`run_id` is “” with no declared input set or algorithm.

Why it matters

T-03b’s `diff` and any dedupe/debugging (§27 provenance) depend on stability AND discrimination.

Potential consequence

Either collisions (missing inputs) or irreproducibility across implementations (unpinned algorithm).

Recommended resolution

Pin: `run_id := sha256(canonical_json(question, corpus_revision, budgets, fault_spec, policy, prompt_version))[:12]`.

F-006 — Contradiction-detection protocol unspecified

Severity: MEDIUM

Location: R-12, E-08, T-08f, C-11 (`contradictory_sources` corpus fixture)

Observation

E-08/T-08f demand that “silently adopting one value fails final-report validation”, but nothing specifies how the validator/drill harness *knows* two retrieved sources assert contradictory values for the same quantity — a semantic judgment the deterministic core cannot perform without a fixture-level marker.

Why it matters

§24 (contradictory sources) is one of the chapter’s central lessons; its acceptance test needs a mechanical trigger.

Potential consequence

Each implementer invents a different detection heuristic, or the drill degenerates to “trust the MockPolicy to populate `conflicts`”.

Recommended resolution

Specify a fixture marker protocol: `contradiction_pair` documents carry `{"conflict_marker": {"quantity": "performance", "with": <doc_id>, "values": [...]}}`; final validation (drill path) requires `report.conflicts` to cover every marker among retrieved docs.

F-007 — `repair_success` pairing rule ambiguous

Severity: MEDIUM

Location: C-10, I-005, T-10

Observation

`repair_success` is “repairs that led to a valid call” with no pairing window: does an `invalid_arguments` observation count as repaired if a valid call to the same tool occurs *any* later step? Different tool? What if the policy changes strategy instead?

Why it matters

The metric is asserted in T-10 against hand-computed values — which are not computable from the spec.

Potential consequence

Two testers compute different values for identical traces.

Recommended resolution

Pin: numerator = invalid-argument observations followed, before episode termination, by a valid call to the **same tool**; denominator = such observations; empty denominator $\rightarrow 1.0$ (I-005).

F-008 — `drill --real` surface unmotivated

Severity: LOW

Location: §5.1 (drill row), R-09

Observation

The drill subcommand lists exit 4 `PULL_REQUIRED` and `[--mock]` as if a real-policy drill existed, while R-09 fixes all drills to `MockPolicy`.

Why it matters

An implementer may build and expose a meaningless `--real` drill mode.

Recommended resolution

State that `drill` forces the `MockPolicy` regardless of `--real` (or forbid the flag), removing exit 4 from the drill row.

F-009 — Exit-code precedence unstated

Severity: LOW

Location: K-03, §5.1, E-02/E-10/E-14

Observation

Usage errors (2), corpus violations (3), and drill verdict fail (1) can co-occur in theory; precedence (usage $\rightarrow$ config $\rightarrow$ corpus $\rightarrow$ episode $\rightarrow$ verdict) is natural but unstated.

Recommended resolution

One sentence in §5.1: “error classes are evaluated in the order usage $\rightarrow$ config $\rightarrow$ corpus $\rightarrow$ episode $\rightarrow$ verdict; the first failing class sets the exit code.”

F-010 — `seen_actions` bookkeeping gaps

Severity: LOW

Location: C-04, C-05 step 6, R-10

Observation

Whether `final` decisions, denied calls, and invalid-argument decisions increment `seen_actions` is unspecified; only the successful-execution case is implied. This changes `repeated_state` behavior for stuck-denied or stuck-invalid policies.

Recommended resolution

Pin: `seen_actions` records every validated-or-denied canonical (`tool`, `arguments`) pair (including denials, excluding `final` decisions and including invalid pairs after canonicalization), so repetition detection covers stuck loops of any class.

F-011 — O-1/O-2 optionality ids unexplained

Severity: LOW

Location: §1 (Policy actor), C-09

Observation

O-1 (MockPolicy double) and O-2 (confirm-effect extension) appear without the family note the ch3/ch4 specs carried.

Recommended resolution

Add the one-line taxonomy footnote (or drop the tags).

F-012 — Drill-report text fields' provenance unstated

Severity: MEDIUM

Location: C-12, I-003, T-03b

Observation

C-12 pins `expected_*` fields but says nothing about `model_behavior` / `runtime_behavior` / `expected_behavior` / `instrumentation` strings: pinned templates per drill, or generated summaries? Byte-identity (I-003) forbids generation without a fixed template.

Recommended resolution

Declare all four fields as per-drill fixed template strings in `drills.py` (with the trace values interpolated only through documented placeholders), or move them out of the graded artifact.

5. Requirements Review

Requirements are observable-behavior-form and uniformly testable (each R maps to named T ids). R-05/R-06/R-07 are exemplary — closed enum, total taxonomy, code-boundary authorization. The weakest rows are R-13 (surrogate formulas — F-002) and R-09 (drill expectations deferred to unpinned `drills.py` contents — F-001). No requirement conflicts were found; no functionality outside the declared scope leaks in (delegation §14 and human-in-the-loop §16 correctly declared out of scope).

6. Interface and Data-Contract Review

Tool contracts (C-01), budgets (C-06), authorization (C-09), and the trace record (C-07) are complete with field-level pinning and drift on the safe side. The gaps: `DECISION_SCHEMA` / `REPORT_SCHEMA` are pseudo-schemas with prose cell values rather than literal shapes (F-003), `ToolSpec.failure_classes` is described but no per-tool class assignment is given (each tool advertises only the classes the fixture raises — worth pinning), and the corpus JSON fixture schema (document fields, conflict markers) is referenced but not defined (F-006).

7. State and Failure Review

The episode state machine and the §3.2 flow diagram are consistent with C-05’s pinned stage order; terminal/failure transitions are explicit. The retry taxonomy is **total** — the strongest failure model in the series so far. Residual gaps: `seen_actions` bookkeeping (F-010) and the `INVALID_INPUT`-vs-decision-error extension in E-07 (works, but belongs in the C-08 table as a decision-class row rather than an edge note).

8. Determinism and Algorithm Review

Search ranking (“lexical overlap, ties broken by `doc_id` sort”) is a sketch rather than a formula; surrogate numerics (F-002) and the `MockPolicy` script (F-004) are the two places where byte-identity is asserted but not currently achievable across implementations. Loop-order and budget-evaluation precedence are correctly pinned.

9. Edge-Case Review

E-01..E-16 is comprehensive on the paths the chapter names (timeout, empty, malformed, contradiction, permission, infinite loop). Notable absences that materially affect conformance: the contradiction-detection trigger (F-006) and exit-precedence (F-009). Everything else unusual is legitimately implementation freedom.

10. Non-Functional Requirement Review

K-02 (60s episode bound) is measurable; K-01/K-03 exit contract is complete modulo precedence; K-04/K-05 (single-writer coupling, byte-format) are excellent. K-02’s “CI soft target” hedge is acceptable per ch3 precedent.

11. Security and Trust-Boundary Review

The trust boundary is exemplary: authorization in code (I-002), closed-world default deny, gold isolation analog via report-validation membership (I-014), and the adversarial T-07b fixture encoding §25’s exact experiment. The single-principal disclaimer is correctly carried. This is the strongest dimension of the spec.

12. Observability and Provenance Review

Trace record carries the full §27 field list with typed reasoning/action/observation entries; artifacts are versioned and schema-gated; `run_id` needs input pinning (F-005). Drill reports preserve the four-question answers — pending F-012.

13. Testing and Verification Review

36 offline test ids + 2 manual smoke ids, each acceptance row traces to requirements. T-09 (self-grading honesty test) is a standout. Gaps: T-08a hedging (F-001), T-10 metric arithmetic (F-007), and the drill/mock-repair expectations (F-004).

14. Metrics and Evaluation Review

LOOP_METRIC_KEYS is a closed, total list with zero-denominator fallbacks — correctly inherited from ch3/ch4 conventions. Two metrics are under-defined: **repair_success** (F-007) and the surrogate numerics feeding **latency_ms/tokens_total/cost_usd_total** (F-002).

15. Traceability Review

The §11 matrix is complete: every R/C/I/K/E/T id routes to module + test. Curriculum § coverage (§1–§36) is literal and exhaustive. This dimension passes at full marks.

16. Internal-Consistency Review

One genuine contradiction found: C-12 “pinned per drill” vs T-08a “(or documented fallback)” (F-001). All carried conventions (E-13 strings, exit codes, schema versioning) checked for verbatim consistency. Terminology is stable except “top hit” / “Lexical overlap” sketches (F-004).

17. Architecture Review

The architecture supports every requirement: the deterministic-core list, the single LLM-import gate, the fault-injection boundary (I-015), and the read-only artifact consumers are coherent and minimal. No redesign recommended.

18. Implementation-Agent Readiness

YES — WITH MINOR CLARIFICATIONS.

Minimum blocking questions (must be pinned before implementation):

1. **Decision JSON canonical shape** (F-003) — discriminator field and **final** payload form.
2. **Per-drill expected termination table** (F-001) — exact **termination_reason** per drill.
3. **Surrogate formulas** (F-002) — tokens/latency/cost derivation.
4. **MockPolicy rule list** (F-004) — decision sequence, repair re-issue, reasoning strings.
5. **Contradiction fixture marker protocol** (F-006) — how the validator triggers E-08 checks.

19. Quality Scorecard

Dimension	Score (0–5)
Scope clarity	5
Terminology	4
Requirement precision	4
Interface completeness	3
Data-contract completeness	4
State/lifecycle definition	4
Algorithm precision	3
Failure semantics	5
Edge-case coverage	4
Non-functional requirements	4
Security specification	5
Observability/provenance	4
Testability	4
Evaluation/metrics	4
Traceability	5

Dimension	Score (0–5)
Internal consistency	4
Architecture consistency	4
Implementation readiness	3

20. Remediation Plan

P0 — Blocking (must resolve before implementation)

- **F-001** Per-drill expected verdict table (kills the T-08a hedge).
- **F-002** Pin surrogate formulas for tokens/latency/cost.
- **F-003** Pin the canonical decision JSON shape.

P1 — Important (resolve before claiming conformance)

- **F-004** MockPolicy rule list (decision sequence, repair, reasoning strings).
- **F-005** `run_id` hash inputs/algorithm.
- **F-006** Contradiction fixture marker protocol.
- **F-007** `repair_success` pairing rule.
- **F-012** Drill-report text fields pinned as templates (or excluded from graded artifact).

P2 — Improvement (may defer)

- **F-008** Forbid/document `--real` on `drill`.
- **F-009** Exit-code precedence sentence.
- **F-010** `seen_actions` bookkeeping pin.
- **F-011** O-1/O-2 taxonomy note.

21. Final Verdict

Specification maturity:

Level 2 - Implementable (P0 pins unblock Level 3)

Implementation readiness:

READY WITH MINOR FIXES

Primary blocker:

Three semantic contracts (decision format, surrogate formulas, drill expectations) are asserted but not pinned, so byte-identity and drill verdicts cannot converge across implementations.

Most important improvement:

Add a per-drill expected-termination table and complete the surrogate/decision pins - the spec's structure needs no other redesign.